

Experiment No. 2

Title: Advanced Class Concepts – Decorators and Magic methods

Aim: Design a dynamic report generator in Python that uses decorators, class methods, and magic methods to customize and format reports. The system should allow users to define report templates and apply various formatting options dynamically.

Theory:

- **Decorators:** Functions that modify the behavior of other functions or methods. They are used to add functionality dynamically.
- **Class Methods:** Methods that are bound to the class and not the instance. They can modify the class state that applies across all instances.
- **Magic Methods:** Special methods in Python (e.g., str, repr, call) that enable customization of object behavior and overloading operators.

Experiment 2 Code:

```
def format_report(func):
    def wrapper(self):
        report = func(self)

        if self.uppercase:
            report = report.upper()

        if self.border:
            report = "=" * 60 + "\n" + report + "\n" + "=" * 60

        return report
    return wrapper
```

```
class Report:
```

```
    templates = {}

    def __init__(self, report_type, fields,
                  uppercase=False, border=False):
        self.report_type = report_type
        self.fields = fields
        self.uppercase = uppercase
        self.border = border

    @classmethod
    def define_template(cls, report_type, fields):
        cls.templates[report_type] = fields

    @classmethod
```

```

def create_report(cls, report_type, values,
                  uppercase=False, border=False):

    fields = cls.templates[report_type]

    return cls(
        report_type,
        dict(zip(fields, values)),
        uppercase,
        border
    )

@format_report
def generate(self):

    report = self.report_type.upper() + "\n\n"

    for field, value in self.fields.items():
        report += field + " : " + value + "\n"

    return report

def __str__(self):
    return self.generate()

def __len__(self):
    return len(self.fields)

print("DYNAMIC REPORT GENERATOR")
print("-" * 30)

report_type = input("Enter report type: ")

number = int(input("Enter number of fields: "))

fields = []

for i in range(number):
    field = input("Enter field " + str(i + 1) + ": ")
    fields.append(field)

Report.define_template(report_type, fields)

values = []

print("\nEnter values:")

```

```
for field in fields:
    value = input(field + " : ")
    values.append(value)

print("\nChoose formatting options:")
print("1. Normal")
print("2. Uppercase")
print("3. Border")
print("4. Uppercase + Border")

choice = int(input("Enter your choice: "))

uppercase = False
border = False

if choice == 2:
    uppercase = True

elif choice == 3:
    border = True

elif choice == 4:
    uppercase = True
    border = True

report = Report.create_report(
    report_type,
    values,
    uppercase,
    border
)

print("\nGENERATED REPORT")
print(report)

print("\nNumber of fields:", len(report))
```

Experiment 2 Output (Screenshot):

Conclusion:

The dynamic report generator demonstrates the use of decorators, class methods, and magic methods to create a flexible and customizable report generation system. This approach enables users to define and apply various formatting options dynamically, enhancing the reusability and maintainability of the code.